AUSPIFY

Bring Future Through Tech

Project Documentation Report

Advanced E-Commerce Website

Project Title	Auspify – E-Commerce Website
Prepared By	Kashish Agnihotri
Technology Stack	HTML5, CSS3, Vanilla JavaScript
Category	Front-End Web Development
Location	Satna, Madhya Pradesh, India

Submitted as part of front-end development practice work

2026

1. Introduction

This document describes the design and development of Auspify, a fully responsive front-end e-commerce web application built to simulate a real-world online storefront under the brand tagline "Bring Future Through Tech." The project was developed as Task 5 (Advanced) of a front-end practice assignment, with the objective of demonstrating practical command over dynamic state management, complex user interface flows, and vanilla JavaScript application logic — without relying on any external framework, library, or backend service.

The project goes beyond static page-building and focuses on the mechanics expected of a production e-commerce experience: a browsable and filterable product catalog, a persistent shopping cart, a multi-step checkout process, and a working order-history system — all running entirely in the browser and surviving a page refresh through client-side storage. The end result is a single-page application that can realistically stand in as the storefront of a small technology retail brand.

2. Objective

The specific goals set for this task, and addressed through this build, were as follows:

- Design and populate a realistic, filterable product catalog across multiple categories.
- Build a persistent shopping cart with quantity control, stock limits and coupon-based discounting.
- Implement a complete, multi-step checkout flow covering shipping, payment and order review.
- Provide an order-management experience, including status tracking and re-order support.
- Ensure all application state — cart, wishlist, orders and theme — survives a page refresh.

3. Tools & Technologies Used

Layer	Technology	Purpose
Markup	HTML5	Semantic structure for catalog, modals, drawers and templates
Styling	CSS3	Responsive Grid & Flexbox layout, custom properties, light/dark theming
Behaviour	Vanilla JavaScript (ES6)	State management, cart logic, checkout wizard, order tracking, filtering
Persistence	Web Storage API	localStorage for cart, wishlist, order history and theme preference
Tooling	Python http.server	Local static file server for testing

No external UI frameworks (such as Bootstrap or Tailwind) or JavaScript libraries were used. The entire experience — including the cart engine, the checkout wizard and the discount calculator — is hand-written, keeping the project dependency-free and runnable directly in a browser without a build step.

4. Project Structure

The project is organised as a flat, self-contained folder so it can be opened or deployed without any build configuration:

```
ecommerce-website/
|-- index.html # Semantic markup, modals, drawers & templates
|-- style.css # Responsive CSS Grid & Flexbox, CSS variables, dark mode
|-- products.js # Product catalog data & coupon configuration
|-- app.js # State management, cart logic, checkout, orders
`-- README.md # Documentation and walkthrough
```

Keeping markup, styling, data and behaviour in four separate files follows the standard separation-of-concerns practice for static sites: index.html is responsible only for content and structure, style.css owns all visual presentation, products.js owns catalog data, and app.js owns application logic and interactivity.

5. Page Sections

The storefront is composed of the following functional sections and overlays, covering the full shopping journey from browsing to order tracking:

#	Section	Purpose
1	Sticky Header	Logo, live search, theme toggle, order-history, wishlist and cart access with live count badges.
2	Hero Banner	Promotional headline, active coupon callout, and a featured "Trending Deal" quick-add card.
3	Category & Filter Bar	Category pills, a live price-range slider and a sort-order dropdown.
4	Product Grid	Responsive card grid showing 12 catalog products with badges, ratings and pricing.
5	Quick View Modal	Full product detail overlay with stock, rating, features and an in-modal add-to-cart stepper.
6	Cart Drawer	Slide-over panel listing cart items, coupon entry and a live order summary.
7	Checkout Wizard	Three-step flow — Shipping, Payment, Review — ending in order placement.
8	Confirmation Modal	Order ID, estimated delivery date and a printable receipt.
9	Orders & Wishlist	Order-history dashboard with status tracking, plus a saved-items wishlist drawer.

6. State, Pricing & Persistence Strategy

Rather than posting to a backend, the application keeps a single in-memory appState object for the cart, wishlist, applied coupon and order history, and mirrors it to localStorage after every mutating action. This lets the entire shopping session — including an in-progress cart and past orders — survive a full page reload, closely simulating what a server-backed cart would feel like to a visitor.

Pricing Rules Summary

Rule	Behaviour
Free shipping	Automatically applied once the cart subtotal reaches $75; otherwise a flat $9.99 shipping fee applies.
Percentage coupon	Reduces the subtotal by a fixed percentage (e.g. AUSPIFY20 → 20% off).
Flat coupon	Reduces the subtotal by a fixed dollar amount, capped at the subtotal itself.
Shipping coupon	Waives the shipping fee regardless of subtotal (e.g. FREESHIP).
Tax	Calculated at 8% of the post-discount (taxable) amount, never on the pre-discount subtotal.

Responsive Behaviour

Breakpoint	Target Devices	Layout Behaviour
< 992px	Tablets, small laptops	Hero collapses to a single column and the featured visual panel is hidden to save space.
< 768px	Large phones, small tablets	Header search is hidden behind the icon bar; checkout and quick-view forms collapse to one column.
< 480px	Standard smartphones	The cart drawer expands to full screen width; filter controls stack vertically.

7. Interactive Features

All interactivity is implemented in app.js using plain DOM APIs, with no external dependencies. The table below summarises the primary behaviours implemented across the application.

Feature	How it works
Live Search	Debounces keystrokes by 250ms before filtering the catalog by title, description and category.
Category & Price Filters	Category pills and a price-range slider combine with the active sort order to re-render the grid.
Quick View	Opens a detail overlay for a product with a quantity stepper, independent of the main grid state.
Cart Engine	addToCart / removeFromCart mutate appState.cart, respect per-item stock limits, and trigger a full summary recalculation.
Coupon Engine	Validates a code against validCoupons and applies percent, flat or shipping-type discounts.
Checkout Wizard	Steps through Shipping → Payment → Review, toggling step panels and validating required fields before advancing.
Order Placement	Generates a unique Order ID and estimated delivery date, then persists the order to history.
Order Tracking	Renders a four-stage progress bar (Placed → Confirmed → Shipped → Delivered) per order, with Buy Again / Cancel actions.
Wishlist	Toggles saved items independent of the cart, with a one-click move-to-cart action.
Theme Toggle	Switches CSS custom-property sets for light/dark mode and persists the preference.
Toast Notifications	Shows short-lived, non-blocking confirmations for cart, wishlist and coupon actions.

8. Implementation Highlight: Cart Total Calculation

One representative piece of logic is the cart pricing engine, which is re-run after every cart or coupon change to keep the summary panel accurate. It is a pure function — given the current cart and applied coupon, it always returns the same totals — which keeps it simple to reason about and easy to unit-test in isolation from the DOM.

```
function calculateCartTotals() {
  const subtotal = appState.cart.reduce((sum, item) => sum + (item.price *
item.quantity), 0);

  // Shipping: free over $75 or with a shipping-type coupon
  let shippingCost = subtotal >= 75 || subtotal === 0 ? 0 : 9.99;
  if (appState.appliedCoupon && appState.appliedCoupon.type === "shipping") {
    shippingCost = 0;
  }

  // Percentage / flat discount
  let discountAmount = 0;
  if (appState.appliedCoupon) {
    if (appState.appliedCoupon.type === "percent") {
      discountAmount = (subtotal * appState.appliedCoupon.value) / 100;
    } else if (appState.appliedCoupon.type === "flat") {
      discountAmount = Math.min(subtotal, appState.appliedCoupon.value);
    }
  }

  const taxableAmount = Math.max(0, subtotal - discountAmount);
  const tax = taxableAmount * 0.08; // 8% tax
  const total = taxableAmount + shippingCost + tax;

  return { subtotal, shippingCost, tax, discountAmount, total };
}
```

Computing tax on the post-discount taxableAmount rather than the raw subtotal keeps the totals consistent with standard retail pricing rules, and returning a plain object keeps the function decoupled from how the summary panel ultimately renders it.

9. Accessibility & Code Quality

- Semantic landmarks (<header>, <main>, <aside>) are used for the header, catalog and cart drawer instead of generic wrappers.
- Interactive controls (drawers, modals, quantity steppers) carry appropriate aria-label attributes for screen-reader clarity.
- Keyboard support is included for every modal and drawer, each of which closes on the Escape key.
- CSS custom properties drive the entire colour palette, keeping the dark-mode implementation and any future re-theming centralised in one place.
- Cart, coupon and checkout logic are written as small, single-purpose functions to keep the codebase easy to trace and extend.

10. Brand Identity

Token	Value	Usage
Primary	`#2563EB`	Buttons, links, active states (Royal Blue)
Secondary	`#06B6D4`	Gradients and hero accents (Cyan)
Tertiary Accent	`#0D9488`	Success and trending highlights (Teal)
Dark Surface	`#0B0F19`	Dark-mode background (Deep Navy)
Tagline	—	"Bring Future Through Tech" — used in the hero banner and cover branding

11. How to Run the Project

Option A — Direct File

Open index.html directly in any modern browser (Chrome, Edge, Firefox or Safari) — no server is required.

Option B — Local Server

```
cd ecommerce-website
python -m http.server 3000
```

Then visit http://localhost:3000 in the browser. Running through a local server is closer to production behaviour and avoids any browser restrictions on local file access.

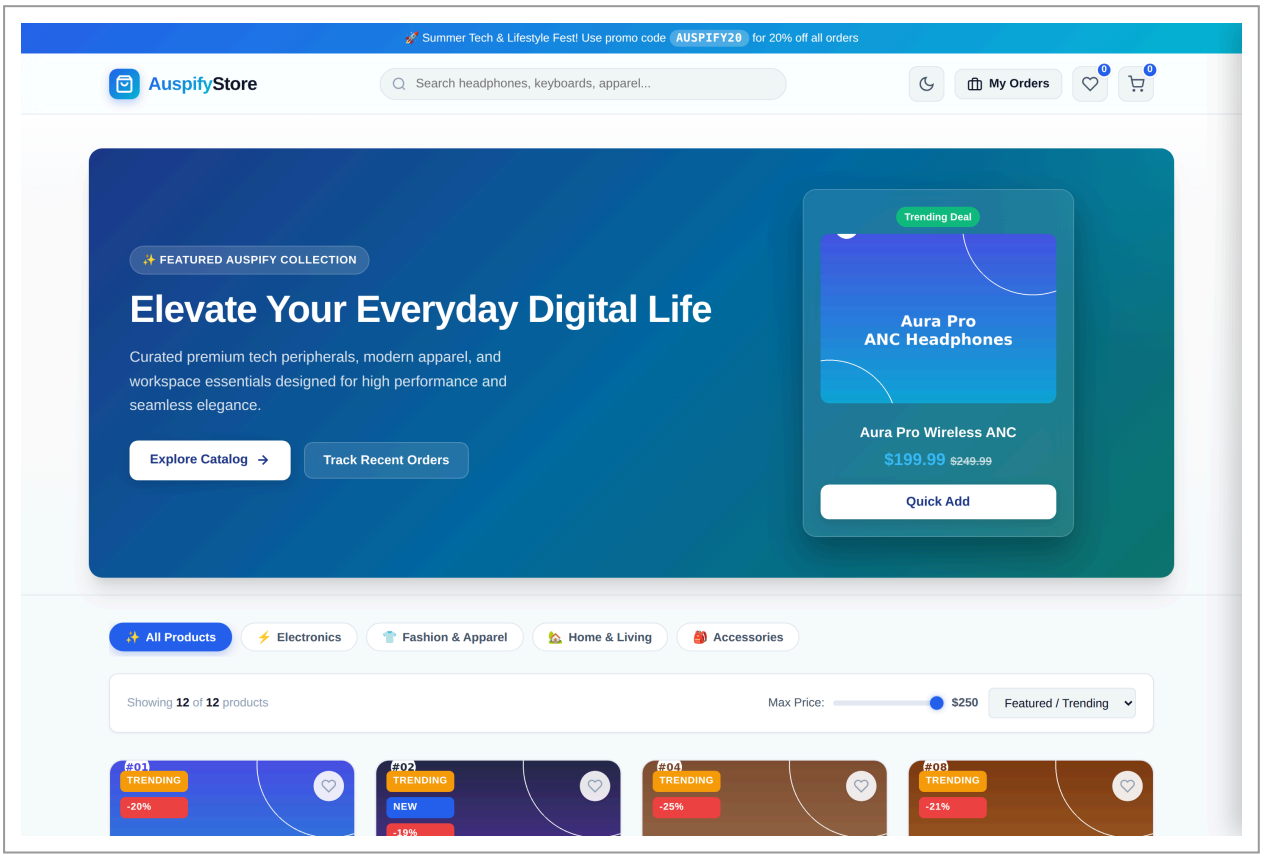
Test Coupon Codes

Code	Type	Effect
AUSPIFY20	Percentage	20% off the entire order
SAVE10	Flat	$10.00 off the order subtotal
FREESHIP	Shipping	Waives standard shipping cost

12. Visual Preview

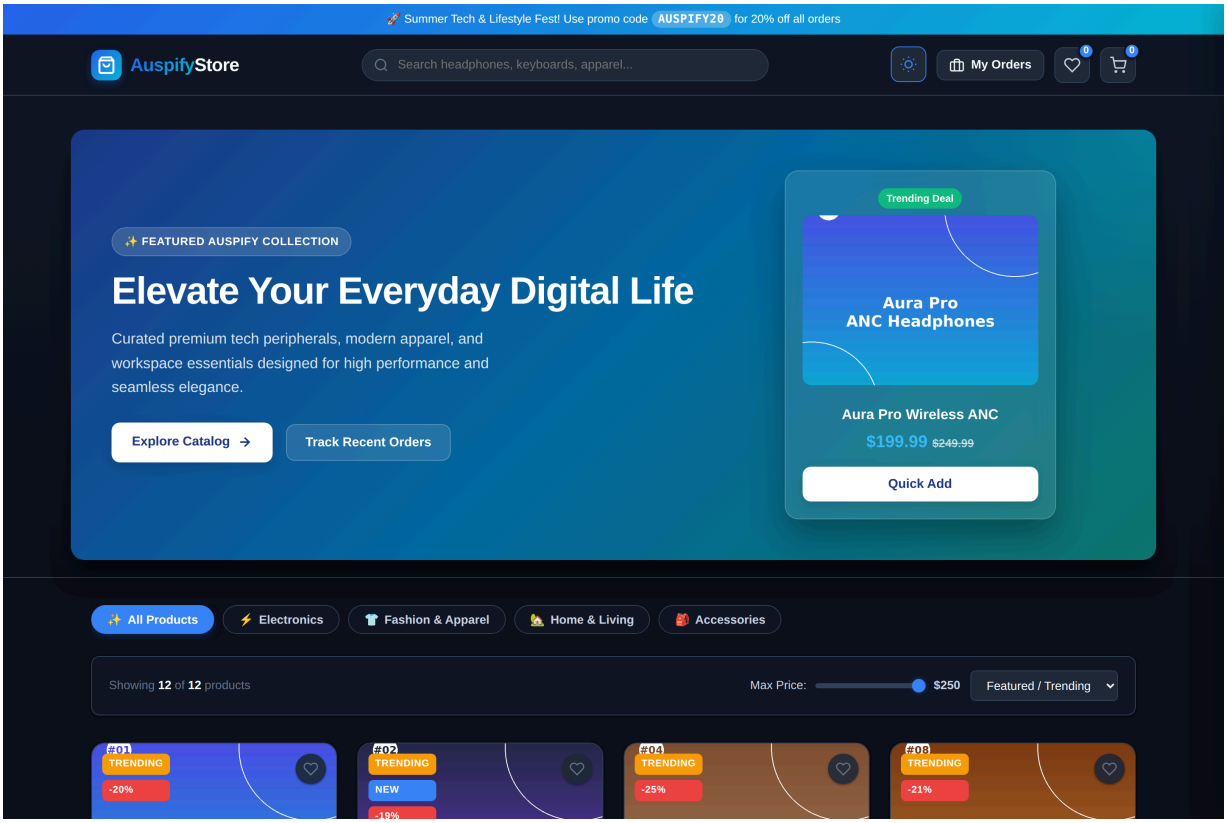
The screenshots below were captured directly from the live index.html file (rendered in a Chromium browser at 1440px width) and show every major flow of the application exactly as a shopper would experience it.

Storefront — Light Mode



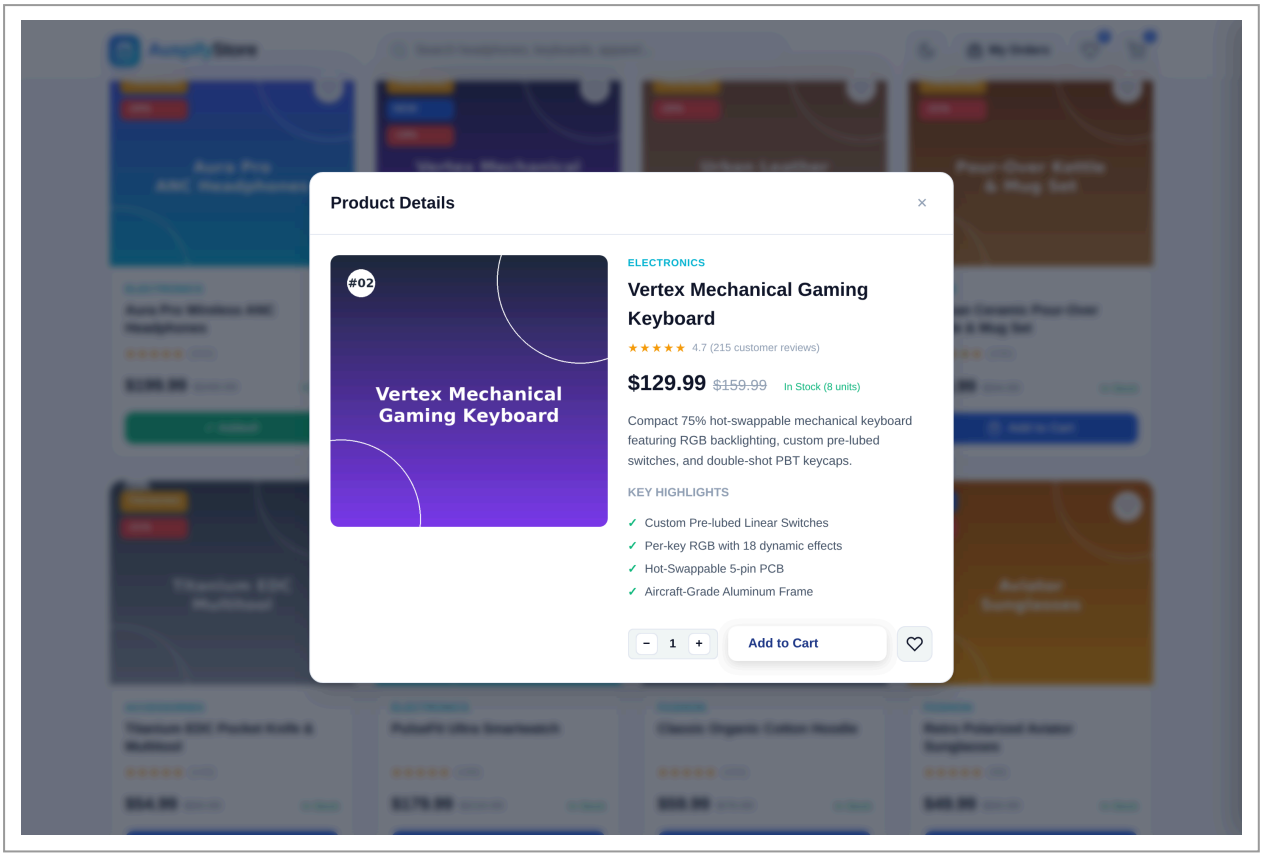
Sticky header, promotional hero banner, category filters and the responsive product grid.

Storefront — Dark Mode



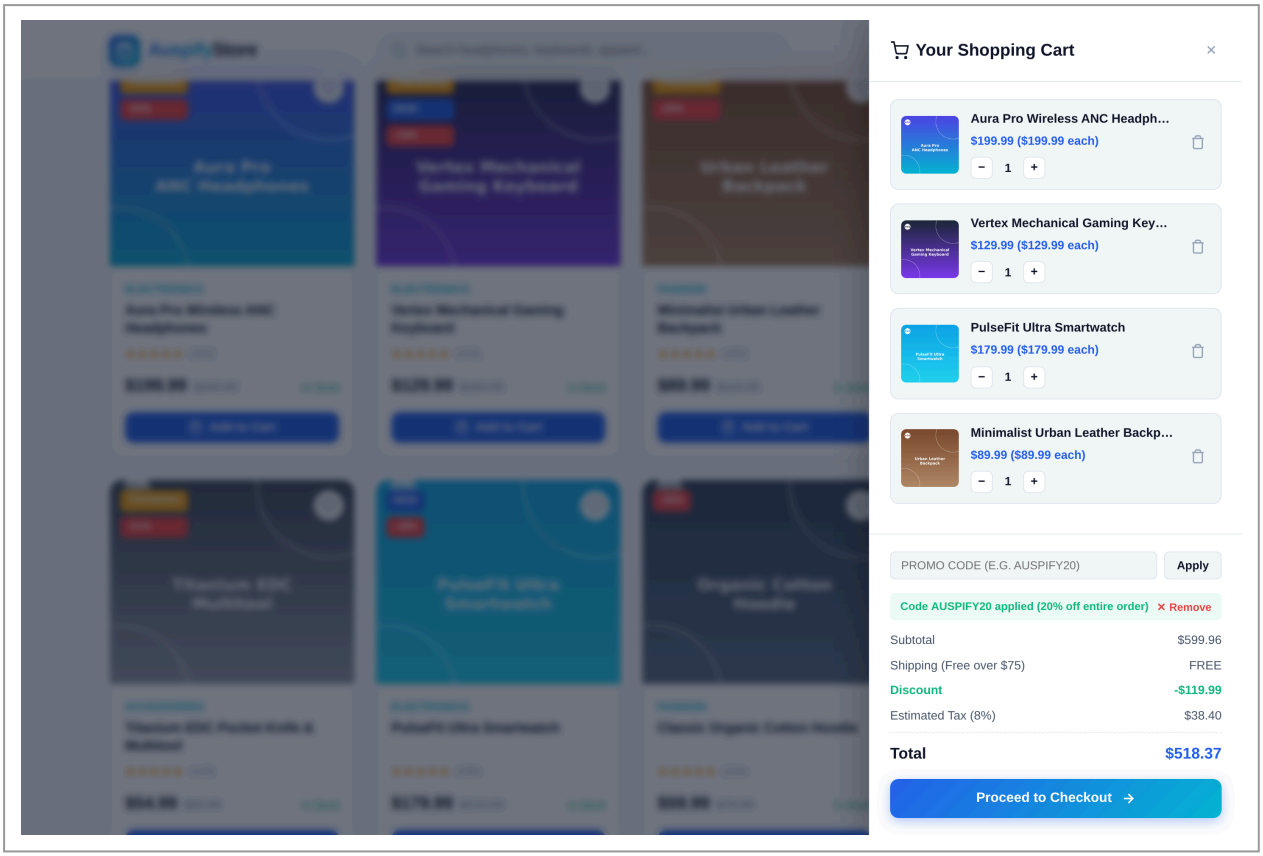
Theme toggle switches the entire storefront to dark mode, with the preference saved via localStorage.

Quick View Modal



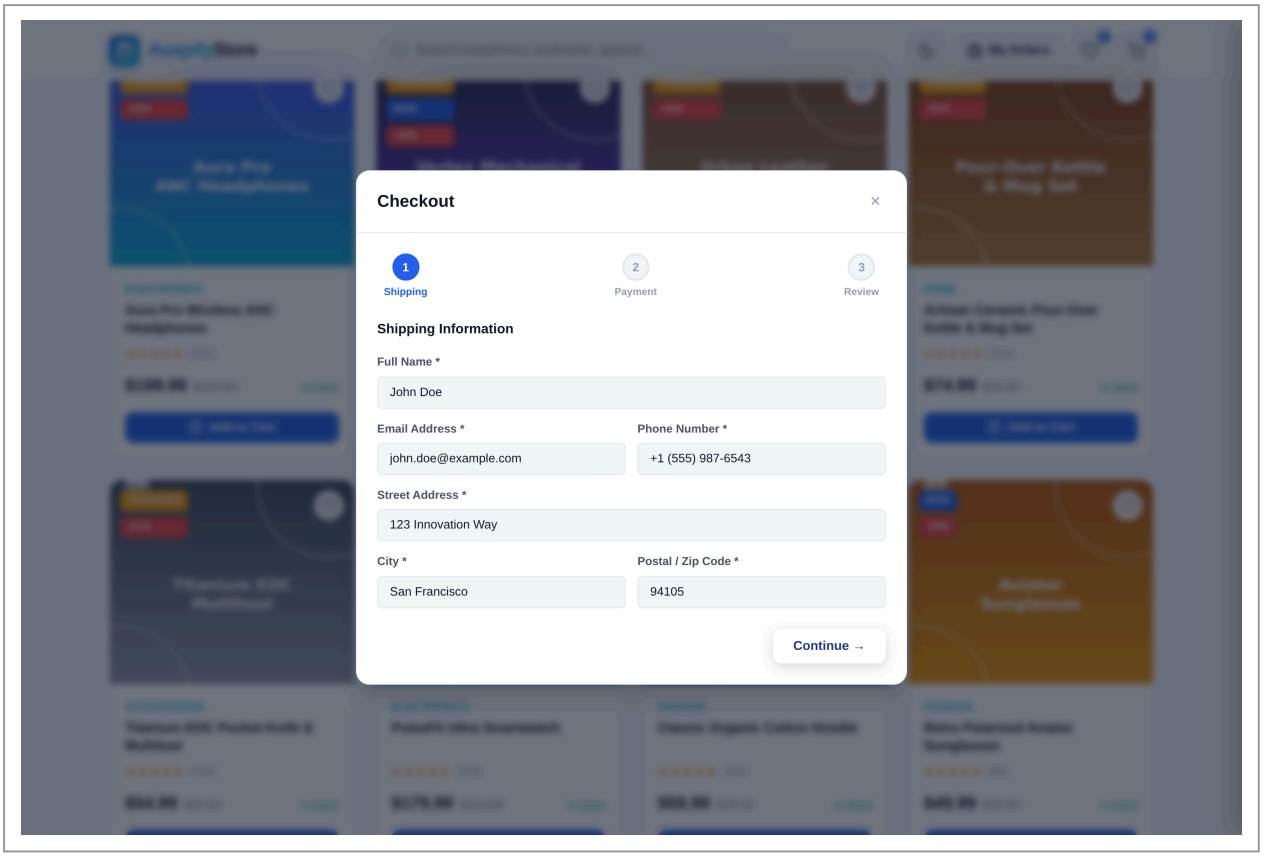
Product detail overlay with rating, stock, feature highlights and an in-modal quantity stepper.

Shopping Cart Drawer



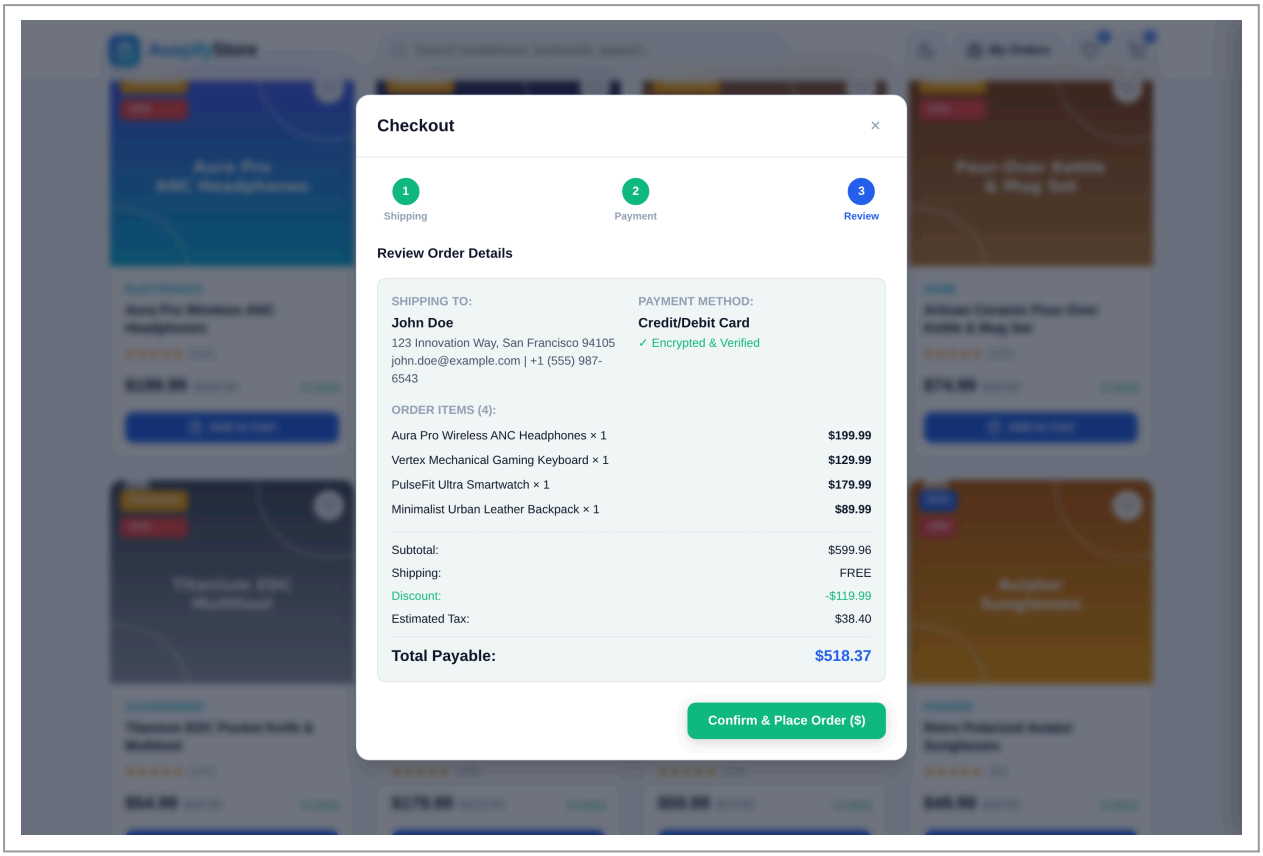
Slide-over cart with live quantity steppers, an applied coupon, and an automatically recalculated summary.

Checkout — Shipping Step



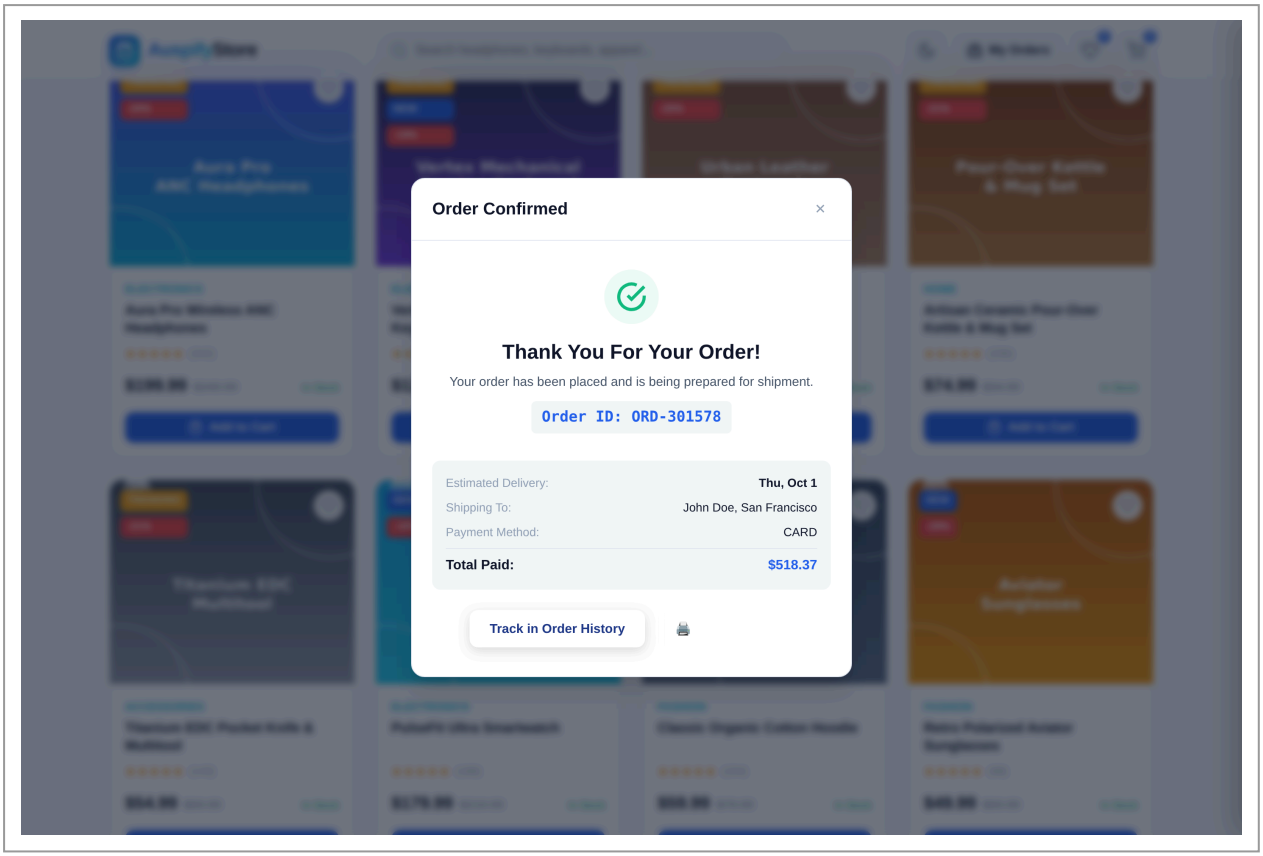
Step one of the checkout wizard, collecting and validating contact and delivery information.

Checkout — Review Step



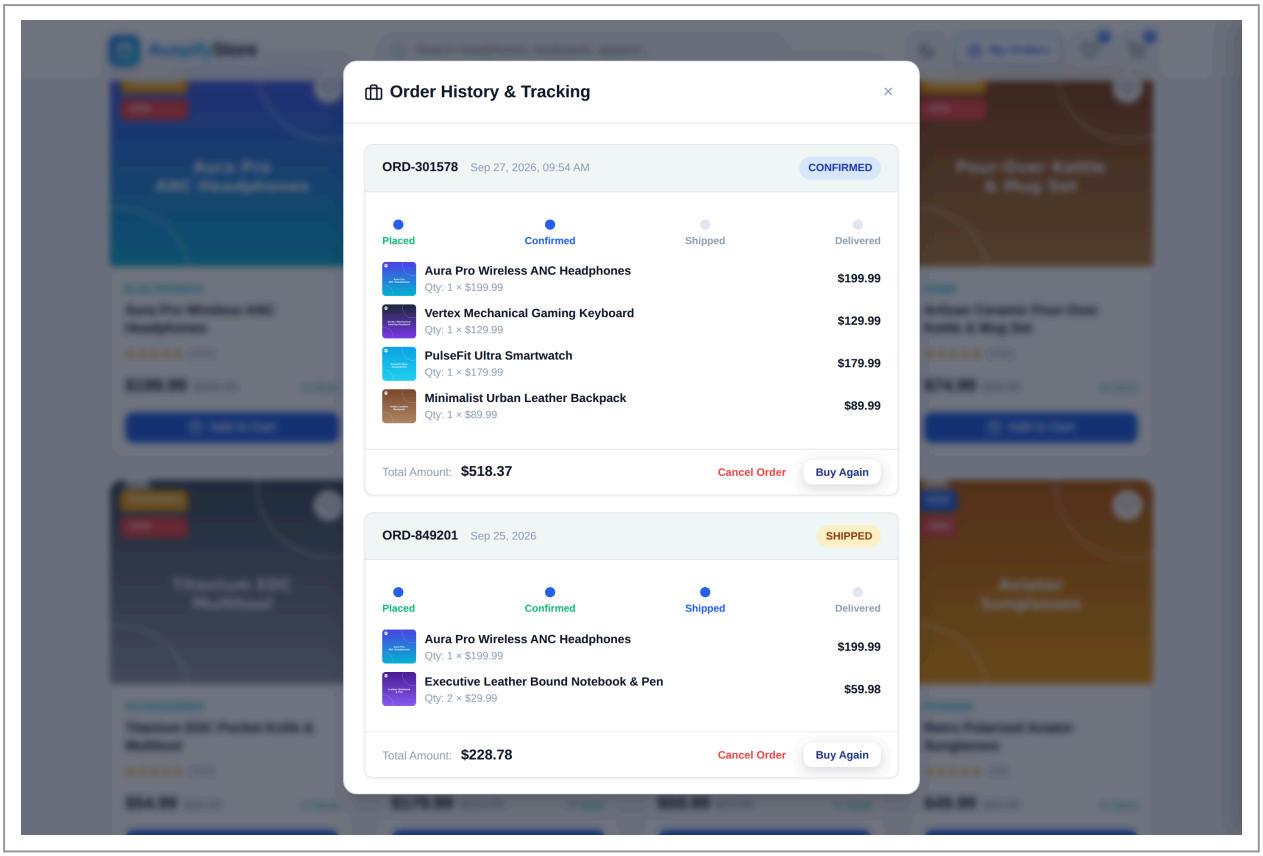
Final review step summarising shipping address, payment method and the full price breakdown.

Order Confirmation



Confirmation modal showing the generated Order ID, estimated delivery date and a printable receipt.

Order History & Status Tracking



Order dashboard with a four-stage tracking bar (Placed → Confirmed → Shipped → Delivered) and Buy Again / Cancel actions.

13. Conclusion

The Auspify e-commerce website meets all five stages of the assigned workflow — catalog design, cart and coupon logic, a multi-step checkout, order management, and full state persistence — while staying entirely framework-free. Because the codebase has zero dependencies, it is easy to extend: additional product categories, payment methods, or order states can be added by following the existing pattern of a state mutation in app.js, a matching render function, and a CSS block driven by the shared custom-property palette. This makes the project a solid, reusable base for future storefront builds beyond the current Auspify brand.

Prepared by Kashish Agnihotri · Satna, Madhya Pradesh · 2026